

Reviewer Pack — Heilmann-Lieb Matching Root Gap Bounds Monotone k-CLIQUE DNF...

Entry 4b37abf22009 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 11:40:32 UTC

Heilmann-Lieb Matching Root Gap Bounds Monotone k-CLIQUE DNF Size

Entry ID: 4b37abf22009 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 11:40:32 UTC

1. Conjecture

Field A (mathematical branch): Statistical mechanics — Heilmann-Lieb monomer-dimer theory: real-rooted matching polynomial of bipartite incidence graphs (Heilmann-Lieb 1972; Godsil identity; Mohar bounds). Rarely applied to monotone circuit/DNF lower bounds (<5 papers). **Field B** (complexity object): Monotone DNF formula size for the k-CLIQUE indicator (Razborov 1985 monotone setting; edges of K_n as variables; target separation: super-polynomial monotone DNF lower bound).

Statement:

For a monotone DNF $F = T_1 \vee \dots \vee T_m$ over the edge set E of K_n , let $G(F)$ be the bipartite term-variable incidence graph (one side = the m terms, other side = the $|E|$ edges, an edge of $G(F)$ joining T_i to e iff $e \in T_i$) and let $M_F(z) = \sum_k (-1)^k \mu_k(G(F)) z^{m+|E|-2k}$ be its matching polynomial, whose roots are real by Heilmann-Lieb. Define $\rho(F) := \min\{|r| : M_F(r) = 0\}$ and $\mu(F) := \log_2(1 + \rho(F)^2)$. Conjecture: there exist absolute constants $c_1, c_2 > 0$ with $c_1 < c_2$ such that (a) for every monotone DNF F of size $m \leq n^{c_1}$ computing any monotone Boolean function, $\mu(F) \leq c_2 \cdot \log_2 n$, while (b) for every monotone DNF F whose terms equal exactly the edge-sets of the k -cliques of K_n with $k = \lfloor \log_2 n \rfloor$, $\mu(F) \geq \sqrt{n} / c_2$ — a single counterexample on either side falsifies the conjecture.

Rationale (proposer's reasoning):

The matching polynomial of a bipartite incidence graph encodes how concentrated or spread the term-variable incidence pattern is, and Heilmann-Lieb forces all of its roots onto the real line so the smallest $|\text{root}|$ is a clean, robust spectral invariant. A monotone DNF representing k-CLIQUE must reuse each edge across exponentially many heavily overlapping clique-terms, which pushes the bipartite incidence graph toward an essentially regular, high-permanent structure whose matching roots are bounded well away from 0. Small poly-size DNFs over n variables produce sparse incidence graphs whose smallest $|\text{root}|$ is forced near 1 by elementary Heilmann-Lieb bounds, so a separation here would yield a structural monotone lower bound technique unconnected to Razborov's approximator method.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d73bcce901cbc54f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds per $n \in \{8, 10, 12, 14, 16, 20, 24, 28, 32, 36, 40\}$: random monotone DNFs ($m = \lceil n^{1.2} \rceil$, $k \in \{2, 3, 4\}$) must yield $\mu(F) \leq 1.5 \cdot \log_2 n$ in $\geq 28/30$ seeds per n ; clique-family DNFs ($k = \lfloor \log_2 n \rfloor$, exact for $n \leq 14$, random $n/2$ -subgraph stand-in for $n > 14$) must yield $\mu(F) \geq 0.5 \cdot \sqrt{n}$ in $\geq 28/30$ seeds per n . Both sides must hold for every n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.85	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -matching polynomial Heilmann-Lieb monotone circuit lower bounds clique
- real-rooted matching polynomial incidence graph DNF size complexity - Razborov k-clique
monotone DNF lower bound matching polynomial roots

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_dnf(n, m, k):
        terms = []
        for _ in range(m):
            term = set()
            while len(term) < k:
                u, v = random.sample(range(n), 2)
                if (u, v) not in term and (v, u) not in term:
                    term.add((u, v))
            terms.append(term)
        return terms
```

```

def matching_polynomial(G, z):
    n1, n2 = len(G[0]), len(G[1])
    M = [[0] * (n1 + n2 + 1) for _ in range(n1 + n2 + 1)]
    M[n1][n2] = 1
    for u in G[0]:
        for v in G[1]:
            if (u, v) not in G:
                continue
            M[u][v] += z * M[u-1][v-1]
            for w in range(v):
                M[w][v] -= M[w][v-1]
    return [M[i][n2] for i in range(n1 + 1)]

def sturm_sequence(poly):
    n = len(poly)
    s = [poly[:]]
    while True:
        if not s[-1]:
            break
        q = []
        r = s[-1]
        while len(r) > 0 and r[-1] != 0:
            d = r[-1]
            q.append([c / d for c in r[:-1]])
            r = [s[-2][-i-1] - (q[-1][-i-1] * s[-1][-i]) for i in range(len(s[-2]) - len(q[-1]))]
        s.append(r)
    return s

def min_root(poly):
    s = sturm_sequence(poly)
    signs = [(-1) ** i * poly[0] for i in range(len(s))]
    sign_changes = sum(signs[i] != signs[i+1] for i in range(len(signs)-1))
    if sign_changes == 0:
        return float('inf')
    else:
        return math.sqrt(2 * (sign_changes - 1))

def mu(F):
    G = ([], [])
    for term in F:
        for edge in term:
            G[0].append(term)
            G[1].append(edge)
    poly = matching_polynomial(G, 1)
    rho = min_root(poly)
    return math.log2(1 + rho**2)

n_values = [8, 10, 12, 14, 16, 20, 24, 28, 32, 36, 40]
results = []

for n in n_values:
    m = math.ceil(n ** 1.2)
    k_values = [2, 3, 4]

    random_dnf = generate_monotone_dnf(n, m, random.choice(k_values))
    random_mu = mu(random_dnf)

```

```

results.append({
    "n": n,
    "m": m,
    "k": random.choice(k_values),
    "type": "random",
    "mu": random_mu
})

if n <= 14:
    clique_family_dnf = generate_monotone_dnf(n, math.comb(n, int(math.log2(n))), int(math.log2(n)))
else:
    subgraph_size = n // 2
    subgraph_edges = [e for e in range(subgraph_size * (subgraph_size - 1) // 2)]
    clique_family_dnf = generate_monotone_dnf(subgraph_size, math.comb(subgraph_size, int(math.log2(subgraph_size))), int(math.log2(subgraph_size)))
    clique_family_mu = mu(clique_family_dnf)
results.append({
    "n": n,
    "m": math.comb(n, int(math.log2(n))),
    "k": int(math.log2(n)),
    "type": "clique-family",
    "mu": clique_family_mu
})

random_support = sum(1 for r in results if r["type"] == "random" and r["mu"] <= 1.5 * math.log2(n))
clique_family_support = sum(1 for r in results if r["type"] == "clique-family" and r["mu"] >= 1.5 * math.log2(n))

support_fraction = (random_support + clique_family_support) / len(results)

if not random_support or not clique_family_support:
    return {
        "metric_name": "mu",
        "metric_value": None,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "support_condition_violated"
    }

mean_mu = sum(r["mu"] for r in results) / len(results)
std_mu = math.sqrt(sum((r["mu"] - mean_mu)**2 for r in results) / len(results))

return {
    "metric_name": "mu",
    "metric_value": mean_mu,
    "instances_tested": len(results),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_mu = sum(r["metric_value"] for r in results) / len(results)
std_mu = math.sqrt(sum((r["metric_value"] - mean_mu)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_mu} std={std_mu} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='support_condition_violated' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_condition_violated")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5733543c.py", line 141, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5733543c.py", line 87, in run_trial
    random_mu = mu(random_dnf)
                ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5733543c.py", line 76, in mu
    rho = min_root(poly)
           ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5733543c.py", line 61, in min_root
    s = sturm_sequence(poly)
        ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5733543c.py", line 56, in sturm_sequence
    r = [s[-2][-i-1] - (q[-1][-i-1] * s[-1][-i]) for i in range(len(s[-2])-len(q[-1]))]
                                     ~~~~~

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in the Sturm sequence routine before producing any data, so neither the support nor falsification condition could be evaluated. | next: Fix the sturm_sequence implementation (correct the index arithmetic in the polynomial remainder step) or replace it with numpy.roots on the matching polynomial coefficients, then re-run.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	205350
2	propose	claude_max	opus	0	0	273273
3	preregistration	claude_max	opus	0	0	7136
4	novelty	claude_max	opus	0	0	3720
5	novelty	claude_max	opus	0	0	7068
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17781
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13043
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17457
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17485
10	judge	claude_max	opus	0	0	4144

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 566456 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/4b37abf22009.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4b37abf22009.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4b37abf22009.tar.gz` (if generated)